Algorithms

GATE CSE 2027 · complete formula, rule, method & trap sheet

50 reference boxes · **9** chapters · **30** worked examples (switch on above)

Asymptotic Analysis & Recurrences | Searching & Sorting | Divide & Conquer | Greedy Algorithms | Dynamic Programming | Graph Algorithms | Amortized Analysis, Hashing & NP-Completeness | Problem → Standard Method Playbook | Last-minute one-liners

1 Asymptotic Analysis & Recurrences

Asymptotic notation

REFERENCE TABLE

Notation	Meaning	Read as
$f = O(g)$	$f(n) \le c \cdot g(n)$ for all $n \ge n_0$	g is an **upper** bound
$f = \Omega(g)$	$f(n) \ge c \cdot g(n)$ for all $n \ge n_0$	g is a **lower** bound
$f = \Theta(g)$	both $O(g)$ and $\Omega(g)$	**tight** bound (same growth rate)
$f = o(g)$ / $\omega(g)$	$f/g \to 0$ / $f/g \to \infty$	**strictly** smaller / larger

RULES & FACTS

- Constants and lower-order terms are dropped: $5n^2 + 3n \log n = \Theta(n^2)$. The base of a logarithm only changes a constant factor, so $\log_2 n = \Theta(\log_{10} n)$; but 2^n and 4^n are **not** Θ of each other (the base of an exponential matters).
- **Worst, average and best case** are different functions of n; $O/\Omega/\Theta$ describe bounds on whichever function you chose. "Insertion sort is $\Omega(n)$" (best case) and "$O(n^2)$" (worst case) are both true.
- Transitivity holds for O, Ω, Θ; Θ is symmetric; $f = O(g) \Leftrightarrow g = \Omega(f)$.

Growth-rate ladder

FORMULA / WORKING

$$1 < \log\log n < \log n < (\log n)^k < n^{\varepsilon} < n < n \log n < n^2 < n^3 < 2^n < n! < n^n$$

RULES & FACTS

- Any polynomial beats any polylog; any exponential beats any polynomial.
- $\mathbf{n^{\log n} = 2^{(\log n)^2}}$ grows faster than every polynomial but slower than 2^n, because $(\log n)^2 = o(n)$.
- **$\log(n!) = \Theta(n \log n)$** (Stirling). Also $\sqrt{n}$ is much bigger than $\log n$; $n^{0.01}$ beats $(\log n)^{100}$ eventually.
- Compare functions by taking logs or by limits: $\lim f/g = 0 \Rightarrow f = o(g)$.

Analysing loops

RULES & FACTS

- `for (i = 1; i < n; i *= 2)` runs $\lceil \log_2 n \rceil$ times. `for (i = 1; i*i <= n; i++)` runs $\lfloor \sqrt{n} \rfloor$ times ($n = 100 \to 10$).
- Nested independent loops multiply: $n \times (\log n) \times \ldots$; when the inner bound depends on the outer index, **sum** the series instead.
- For `for (i = n; i > 1; i = √i)` the count is $\Theta(\log \log n)$.
- Space complexity counts extra memory beyond the input, including the **recursion stack depth**.

Solving recurrences

Master theorem for $T(n) = a \cdot T(n/b) + f(n)$, $a \ge 1$, $b > 1$

NOTES

Let $n^c = n^{\log_b a}$ (the work done at the leaves). Compare f(n) with it:

REFERENCE TABLE

Case	Condition	Result
1	$f(n) = O(n^{c-\varepsilon})$ — leaves dominate	$T(n) = \Theta(n^c)$
2	$f(n) = \Theta(n^c \log^k n)$, $k \ge 0$ — balanced	$T(n) = \Theta(n^c \log^{k+1} n)$
3	$f(n) = \Omega(n^{c+\varepsilon})$ and $a \cdot f(n/b) \le \delta \cdot f(n)$, $\delta < 1$ — root dominates	$T(n) = \Theta(f(n))$

REFERENCE TABLE

Recurrence	$n^{\log_b a}$	Case	Solution
$T(n) = 2T(n/2) + n$ *(merge sort)*	n	2	**$\Theta(n \log n)$**
$T(n) = 4T(n/2) + n$	n^2	1	$\Theta(n^2)$
$T(n) = 4T(n/2) + n^2$	n^2	2	**$\Theta(n^2 \log n)$**
$T(n) = 3T(n/2) + n^2$	$n^{1.58}$	3	$\Theta(n^2)$
$T(n) = T(n/2) + 1$ *(binary search)*	1	2	$\Theta(\log n)$
$T(n) = 2T(n/2) + 1$	n	1	$\Theta(n)$
$T(n) = 2T(n/2) + n \log n$	n	2 ($k = 1$)	**$\Theta(n \log^2 n)$**
$T(n) = 7T(n/2) + n^2$ *(Strassen)*	$n^{2.81}$	1	$\Theta(n^{\log_2 7})$

COMMON TRAP

When the Master theorem does not apply

a or b not constant, f(n) not polynomially related (e.g. n/log n), or subproblems of unequal size such as T(n/3) + T(2n/3). Use the recursion tree, substitution, or the Akra–Bazzi idea instead.

Other forms

REFERENCE TABLE

Recurrence	Solution	How
$T(n) = T(n-1) + n$	$\Theta(n^2)$	unroll: $n + (n-1) + \dots + 1$
$T(n) = T(n-1) + 1$	$\Theta(n)$	unroll
$T(n) = 2T(n-1) + 1$	$\Theta(2^n)$	$T(n) + 1$ doubles each step (Tower of Hanoi: $2^n - 1$)
$T(n) = T(n-1) + T(n-2) + 1$	$\Theta(\varphi^n)$	Fibonacci-type, exponential
$T(n) = T(\sqrt{n}) + 1$	$\Theta(\log \log n)$	$n = 2^{(2^k)}$ takes k steps
$T(n) = T(n/3) + T(2n/3) + n$	$\Theta(n \log n)$	recursion tree: every level costs $\le n$, depth $\log_{3/2} n$ (checked numerically)
$T(n) = T(n/5) + T(7n/10) + n$	$\Theta(n)$	sizes sum to $9n/10 < n$: geometric decrease (median of medians)

2 Searching & Sorting

Searching

RULES & FACTS

- **Linear search**: $\Theta(n)$ worst case, $(n+1)/2$ comparisons on average for a successful search.
- **Binary search** on a sorted array: at most $\lfloor \log_2 n \rfloor + 1$ comparisons ($n = 15 \to 4$, $n = 1000 \to 10$ because $2^{10} = 1024$). Recurrence $T(n) = T(n/2) + 1$. Not possible in $O(\log n)$ on a linked list.
- Interpolation search: $O(\log \log n)$ on uniformly distributed data, $O(n)$ worst case. Hash search: $O(1)$ average.

Comparison sorts at a glance

REFERENCE TABLE

Algorithm	Best	Average	Worst	Extra space	Stable?
Bubble	n (with early exit)	n^2	n^2	1	Yes
Selection	n^2	n^2	n^2	1	No
Insertion	**n**	n^2	n^2	1	Yes
Merge	n log n	n log n	n log n	O(n)	Yes
Quick	n log n	n log n	**n^2**	O(log n) stack (O(n) worst)	No
Heap	n log n	n log n	n log n	1	No

NOTES

A sort is **stable** if equal keys keep their original relative order, and **in place** if it uses O(1) extra memory. Selection sort makes at most $n-1$ swaps (the fewest of the simple sorts); insertion sort is best for nearly sorted data and small arrays; merge sort is the choice for linked lists and external sorting.

Lower bound and non-comparison sorts

RULES & FACTS

- Any comparison sort corresponds to a binary **decision tree** with at least $n!$ leaves, so its height is $\ge \lceil \log_2 n! \rceil = \Omega(n \log n)$. Sorting 5 items needs at least $\lceil \log_2 120 \rceil$ = **7** comparisons in the worst case.
- **Counting sort**: keys are integers in $0 \dots k$; $\Theta(n + k)$ time and space, stable. **Radix sort**: d passes of a stable sort, $\Theta(d(n + k))$. **Bucket sort**: $\Theta(n)$ expected for uniform input. These beat n log n only because they do not compare keys.

Counting operations

FORMULA / RULE

Inversions

A pair $i < j$ with $a[i] > a[j]$. Maximum $n(n-1)/2$ (reverse sorted). Insertion sort performs **exactly one shift per inversion**; bubble sort performs exactly one swap per inversion.

FORMULA / RULE

Useful exact counts

```
merge two sorted lists (m, n): m + n − 1 worst
merge sort, n = 2^k:  n log₂n − n + 1  worst
selection sort comparisons: n(n−1)/2
max AND min together:  ⌈3n/2⌉ − 2
second largest:       n + ⌈log₂n⌉ − 2
```

Selection problems

RULES & FACTS

- **Quickselect** (k-th smallest): partition around a pivot and recurse into one side — O(n) average, $O(n^2)$ worst.
- **Median of medians** guarantees a good pivot (groups of 5), giving O(n) worst case: $T(n) = T(n/5) + T(7n/10) + O(n)$.
- Finding the maximum needs exactly $n - 1$ comparisons. Finding max and min together takes $\lceil 3n/2 \rceil - 2$ (compare in pairs first): $n = 8 \to 10$, $n = 100 \to$ **148** (verified by simulation).

GATE FAVOURITE

GATE angle

Know which sort is stable, in place, adaptive (insertion), and which has an $O(n^2)$ worst case (quick, insertion, bubble, selection). Expect questions on **number of swaps/comparisons for a given array**, **effect of the pivot rule** and "which sorting algorithm will take the least time on an already sorted input" (insertion sort: $\Theta(n)$).

3 Divide & Conquer

The pattern

RULES & FACTS

1. **Divide** the input into a smaller instances of size n/b.
2. **Conquer** by solving them recursively (base case for tiny inputs).
3. **Combine** the answers in f(n) time.

REFERENCE TABLE

Algorithm	Recurrence	Time
Binary search	$T(n) = T(n/2) + 1$	$\Theta(\log n)$
Merge sort	$T(n) = 2T(n/2) + n$	$\Theta(n \log n)$
Quicksort (balanced split)	$T(n) = 2T(n/2) + n$	$\Theta(n \log n)$; worst $\Theta(n^2)$
Max & min of an array	$T(n) = 2T(n/2) + 2$	$3n/2 - 2$ comparisons (n a power of 2)
Maximum-subarray (D&C)	$T(n) = 2T(n/2) + n$	$\Theta(n \log n)$ (Kadane's DP does $\Theta(n)$)
Closest pair of points	$T(n) = 2T(n/2) + n$	$\Theta(n \log n)$
Karatsuba multiplication	$T(n) = 3T(n/2) + n$	$\Theta(n^{\log_2 3}) \approx n^{1.585}$
Strassen matrix multiplication	$T(n) = 7T(n/2) + n^2$	$\Theta(n^{\log_2 7}) \approx n^{2.81}$ (naive: n^3)
Median of medians selection	$T(n) = T(n/5) + T(7n/10) + n$	$\Theta(n)$
Fast exponentiation	$T(n) = T(n/2) + 1$	$\Theta(\log n)$ multiplications

Worked ideas

RULES & FACTS

- **Merge step of merge sort** uses O(n) extra space; sorting an array of n elements has recursion depth $\log_2 n$.
- **Counting inversions** in O(n log n): piggy-back on merge sort — when an element from the right half is merged before elements of the left half, it forms inversions with all remaining left elements.
- Efficiency of D&C comes from the combine cost: if the subproblems overlap (e.g. Fibonacci), plain D&C is exponential and **dynamic programming** is needed instead.

4 Greedy Algorithms

When greedy works

RULES & FACTS

- **Greedy-choice property**: some optimal solution contains the greedy first choice (proved by an **exchange argument**: swap any optimal solution's first choice for the greedy one without making it worse).
- **Optimal substructure**: after the greedy choice, the rest is a smaller instance of the same problem.
- Greedy algorithms are usually simple and fast (often O(n log n) because of a sort or a heap) but can be **wrong**: always look for a counter-example.

Classic problems

REFERENCE TABLE

Problem	Greedy rule	Time
Activity selection (maximum number of non-overlapping activities)	sort by **earliest finish time**; take each activity that starts after the last chosen one ends	O(n log n)
Fractional knapsack	sort by value/weight ratio; take whole items, then a fraction of the next	O(n log n)
Huffman coding	repeatedly merge the two least-frequent nodes	O(n log n) with a min-heap
Optimal merge pattern	always merge the two smallest files (same as Huffman)	O(n log n)
Job sequencing with deadlines	sort by profit; place each job in the latest free slot ≤ its deadline	$O(n^2)$ (O(n log n) with union-find)
Minimising total waiting time	shortest job first	O(n log n)
MST (Kruskal, Prim)	cheapest safe edge	see graph section
Dijkstra	closest unvisited vertex	see graph section

5 Dynamic Programming

Method

RULES & FACTS

1. Define the subproblem (the state) and what the table entry means.
2. Write the recurrence that builds a state from smaller ones, plus the base cases.
3. Fill the table in an order that respects dependencies (**tabulation**, bottom-up) or recurse with a cache (**memoisation**, top-down).
4. Time = (number of states) × (work per state); space = table size (often reducible to one or two rows).

NOTES

DP vs divide & conquer: D&C subproblems are independent; DP subproblems overlap. **DP vs greedy**: DP examines all choices (guaranteed optimal when the structure holds); greedy commits to one. Naive recursion on overlapping subproblems is exponential — Fibonacci with memoisation drops from $\Theta(\varphi^n)$ to $\Theta(n)$.

Standard problems

REFERENCE TABLE

Problem	Recurrence	Time / space
Longest common subsequence (strings of length m, n)	L[i][j] = L[i−1][j−1] + 1 if $x_i = y_j$, else max(L[i−1][j], L[i][j−1])	Θ(mn) time; (m+1)(n+1) table
Edit distance	D[i][j] = min(D[i−1][j] + 1, D[i][j−1] + 1, D[i−1][j−1] + [$x_i \neq y_j$])	Θ(mn)
0/1 knapsack (n items, capacity W)	K[i][w] = max(K[i−1][w], v_i + K[i−1][w − w_i])	Θ(nW) — *pseudo-polynomial*
Matrix-chain multiplication	M[i][j] = $\min_{i \le k < j}$ (M[i][k] + M[k+1][j] + $p_{i-1}p_kp_j$)	Θ(n^3) time, Θ(n^2) space
Longest increasing subsequence	L[i] = 1 + max L[j] over j < i with a[j] < a[i]	Θ(n^2); Θ(n log n) with binary search
Coin change (minimum coins)	C[x] = 1 + min C[x − c]	Θ(amount · #coins)
Coin change (number of ways)	for each coin, W[x] += W[x − c]	Θ(amount · #coins)
Subset sum / partition	reachable[s] |= reachable[s − a_i]	Θ(n · sum)
Floyd–Warshall (all-pairs shortest path)	$d^{(k)}[i][j] = \min(d^{(k-1)}[i][j], d^{(k-1)}[i][k] + d^{(k-1)}[k][j])$	Θ(V^3)
Rod cutting, optimal BST, Catalan numbers, longest palindromic subsequence	same pattern: states over intervals or prefixes	

GATE FAVOURITE

GATE angle

Typical asks: the **time or space** of a DP, the **table size**, filling a few cells by hand, whether a problem has overlapping subproblems, and "which of these is solved by DP vs greedy". Remember that 0/1 knapsack and subset sum are **pseudo-polynomial** (polynomial in the numeric value W, not in its bit length), so they do not contradict NP-completeness.

6 Graph Algorithms

Traversal: BFS and DFS

RULES & FACTS

- **BFS** (queue): visits vertices in increasing distance (number of edges) from the source; gives shortest paths in **unweighted** graphs; Θ(V + E) with adjacency lists, Θ(V^2) with a matrix. A graph is **bipartite** iff BFS can 2-colour it (no odd cycle).
- **DFS** (stack / recursion): records discovery and finish times; Θ(V + E). Edge types in a directed graph: **tree, back, forward, cross**. A directed graph has a cycle **iff DFS finds a back edge**; an undirected graph has a cycle iff DFS meets a visited vertex that is not the parent.
- A DFS forest of an undirected graph with c components has V − c tree edges. **Articulation points** and **bridges** are found with DFS low-link values in Θ(V + E).

Topological sort and SCC

RULES & FACTS

- A topological ordering (every edge u → v has u before v) exists **iff the graph is a DAG**. Two methods, both Θ(V + E): (1) **Kahn** — repeatedly remove a vertex of in-degree 0; (2) DFS — output vertices in **decreasing finish time**. The ordering is usually **not unique**.
- **Strongly connected components**: Kosaraju (DFS on G, then DFS on the transpose in decreasing finish time) or Tarjan (one DFS with low-links), both Θ(V + E). Contracting each SCC gives a DAG.

Minimum spanning tree

RULES & FACTS

- An MST of a connected weighted graph has V − 1 edges and minimum total weight. If all edge weights are **distinct**, the MST is **unique**.
- **Cut property**: for any cut, the lightest crossing edge belongs to some MST. **Cycle property**: the heaviest edge on a cycle (if unique) belongs to no MST.
- **Kruskal**: sort edges, add the next edge if it joins two different components (union-find). Θ(E log E) = Θ(E log V).
- **Prim**: grow one tree from a start vertex, always adding the cheapest edge leaving it. Θ(E log V) with a binary heap; Θ(V^2) with an adjacency matrix and no heap; Θ(E + V log V) with a Fibonacci heap.
- Adding the same constant to every edge, or squaring all positive weights, does not change which spanning tree is minimum (only the ordering of weights matters).

Single-source shortest paths

REFERENCE TABLE

Algorithm	Handles	Time
BFS	unweighted graphs	Θ(V + E)
Dijkstra	non-negative weights only	Θ((V + E) log V) with a binary heap; Θ(V^2) with an array
Bellman–Ford	negative edges; **detects negative cycles**	Θ(VE)
DAG relaxation in topological order	negative edges, DAG only	Θ(V + E)
Floyd–Warshall	all pairs; negative edges but no negative cycle	Θ(V^3), Θ(V^2) space
Johnson's algorithm	all pairs on sparse graphs with negative edges	Θ(VE + V^2 log V)

RULES & FACTS

- Relaxation: if d[u] + w(u, v) < d[v], set d[v] = d[u] + w(u, v). Bellman–Ford relaxes every edge V − 1 times; a further improving relaxation means a negative cycle.
- **Dijkstra fails with negative edges** because a vertex's distance is finalised when it is extracted, and a later negative edge could still shorten it.

GATE FAVOURITE

GATE angle

Know the **time complexity** of each algorithm (and its dependence on the data structure), **when it fails**, the order in which Dijkstra extracts vertices, and what the first few Kruskal/Prim steps pick. Floyd–Warshall detects a negative cycle by a negative value on the diagonal.

7 Amortized Analysis, Hashing & NP-Completeness

Amortized analysis

NOTES

The amortized cost of an operation is the **average over a worst-case sequence** — no probability involved. Three methods: **aggregate** (total ÷ n), **accounting** (charge extra credit to cheap operations), **potential** (a potential function $\Phi \geq 0$, amortized cost = actual + $\Delta\Phi$).

RULES & FACTS

- **Union–find** with union by rank and path compression: m operations cost $O(m \cdot \alpha(n))$, where α is the inverse Ackermann function (≤ 4 for any practical n).
- Stack with **multipop**, queue from two stacks, and splay trees are standard amortized examples.

Hashing — analysis

RULES & FACTS

- With chaining and simple uniform hashing, load factor $\alpha = n/m$: expected unsuccessful search $\Theta(1 + \alpha)$, successful $\approx 1 + \alpha/2$. With $\alpha = O(1)$, operations are O(1) expected.
- Open addressing, uniform probing: expected probes $\approx 1/(1 - \alpha)$ for an unsuccessful search and $(1/\alpha) \cdot \ln(1/(1 - \alpha))$ for a successful one.
- A **universal family** of hash functions makes the expected number of collisions small regardless of the input. The **birthday paradox**: collisions are likely once about $\sqrt{m}$ keys are inserted. Cuckoo hashing gives worst-case O(1) lookups.

P, NP and NP-completeness

REFERENCE TABLE

Class	Definition
P	decision problems solvable in polynomial time by a deterministic algorithm
NP	problems whose "yes" answers can be **verified in polynomial time** given a certificate (equivalently, solvable by a non-deterministic polynomial-time machine). P ⊆ NP.
NP-hard	at least as hard as every problem in NP (every NP problem reduces to it); need not be in NP
NP-complete	in NP **and** NP-hard

RULES & FACTS

- **Polynomial-time reduction** $A \leq_p B$: an easy translation of instances of A into instances of B. If B is easy then A is easy; if A is hard then B is hard. To prove B NP-hard, reduce a **known NP-complete** problem **to** B (direction matters!).
- **Cook–Levin theorem**: SAT is NP-complete (the first one). If any NP-complete problem is in P, then **P = NP**; if any is proved to need super-polynomial time, then P ≠ NP.

REFERENCE TABLE

NP-complete	In P (polynomial time)
SAT, **3-SAT**, Clique, Independent Set, Vertex Cover, Hamiltonian cycle/path, Travelling Salesman (decision), Subset Sum, Partition, 3-colouring, 0/1 knapsack (decision), Set cover	**2-SAT**, shortest paths, MST, matching, maximum flow, 2-colouring (bipartiteness), Euler circuit, linear programming, primality testing (AKS), sorting, searching

RULES & FACTS

- **Pseudo-polynomial** algorithms run in time polynomial in the numeric *value* of the input (knapsack $\Theta(nW)$); NP-complete problems that have them are only "weakly" NP-complete.
- **Approximation**: vertex cover has a simple 2-approximation (take both endpoints of a maximal matching); metric TSP has a 2-approximation (double the MST) and Christofides' 1.5.
- Relationship of classes believed true: P ⊂ NP ⊂ PSPACE ⊂ EXPTIME (P ≠ NP is an open question). Co-NP contains the complements of NP problems; P ⊆ NP ∩ co-NP.

GATE FAVOURITE

GATE angle

Typical questions: "which of these is not known to be NP-complete / is in P" (2-SAT, shortest path, MST are in P; 3-SAT, Clique, Hamiltonian cycle, Subset Sum are NPC), "what follows if X is in P", and which direction a reduction must go. The decision version of a problem is the one that is NP-complete; the optimisation version is NP-hard.

8 Problem → Standard Method Playbook

METHOD

Heap Sort — canonical procedure

1. Build a **max-heap** for ascending order (bottom-up heapify, O(n)).
2. Swap the root with the last element of the active heap.
3. Reduce heap size by one and max-heapify the root.
4. Repeat until one element remains.

Time: **O(n log n)** in best/average/worst cases. Auxiliary array space: **O(1)** for in-place heap sort. It is not stable.

METHOD

Open addressing — double hashing

For table size m, the standard probe sequence is:

$h(k,i) = (h_1(k) + i \cdot h_2(k)) \bmod m$, i = 0,1,2,…

- Choose $h_2(k)$ relatively prime to m so the sequence can visit the entire table.
- Do not confuse double hashing with linear probing: linear probing adds a fixed step of 1.
- Deletion normally uses a tombstone/deleted marker; simply clearing a slot can break future searches.

METHOD

BFS / DFS — recognition and invariants

- **BFS:** queue; explores by nondecreasing distance from the source. In an unweighted graph, first discovery gives a shortest-path distance.
- **DFS:** recursion/explicit stack; timestamps classify edges and support cycle detection, topological ordering, SCCs and articulation/bridge algorithms.
- For an undirected graph, DFS produces tree/back edges; a cross-edge classification from directed DFS should not be imported blindly.

METHOD

Articulation point — Tarjan low-link test

During DFS, maintain `disc[u]` and `low[u]`. For a DFS tree edge u → v:

- If u is not the root: **u is an articulation point iff some child v has low[v] ≥ disc[u]**.
- If u is the DFS root: it is an articulation point iff it has at least **2 DFS children**.
- For an undirected graph, a bridge is detected by `low[v] > disc[u]`.

METHOD

Greedy vs DP — selection test

Before choosing an algorithm, ask:

1. Does a locally optimal choice provably extend to a global optimum? → seek a **greedy-choice/exchange proof**.
2. Do many subproblems overlap and can an optimal solution be decomposed into optimal subsolutions? → use **DP**.
3. If neither property is apparent, do not force greedy or DP; consider graph search, divide-and-conquer, backtracking, or another formulation.

9 Last-minute one-liners

LAST-MINUTE · FORMULAS & RULES

- **Master theorem:** $T(n)=aT(n/b)+f(n)$; compare $f(n)$ with $n^{\log_b a}$ before choosing the case.
- **Merge sort:** O(n log n) time and O(n) auxiliary space in the usual array implementation.
- Dijkstra is valid with non-negative edge weights; it is not a general shortest-path algorithm for negative edges.
- Union-Find with path compression + union by rank/size has near-constant amortized cost: $O(\alpha(n))$.

LAST-MINUTE · MUST REMEMBER

- For nested loops, derive the exact iteration count when possible; do not assume every nested loop is $O(n^2)$.
- Binary search requires a sorted search space; its standard comparison complexity is O(log n).
- Greedy algorithms require a proof of correctness, typically via greedy-choice and/or exchange arguments.
- DP is appropriate when subproblems overlap and the problem has optimal substructure.
- A DAG has a topological ordering iff it is acyclic; DFS finishing times can produce one.
- Hash-table performance depends on load factor and collision handling; worst-case search can still be O(n).
- **Heap sort:** build heap in O(n), then perform $n-1$ extract-max operations; O(n log n) time and O(1) auxiliary array space.
- BFS gives shortest paths in unweighted graphs; DFS supplies timestamps/low-link structure for cycle, SCC, bridge and articulation problems.

LAST-MINUTE · COMMON TRAPS

- Quicksort has $O(n^2)$ worst-case time; average/expected time is O(n log n) under standard randomisation assumptions.

LAST-MINUTE · GATE FAVOURITES

- Know the difference between MST algorithms, shortest-path algorithms, and traversal algorithms; they solve different problems.
- **Articulation:** non-root u is critical when some DFS child v has $low[v] \geq disc[u]$; DFS root needs at least two children.